

Mocking with unittest.mock: Mock, MagicMock, and Patching

SE 413/513 — Software Testing

Course Reading Material

Unit: Test Doubles and Mocking

© Dr. Kamrul Hasan, Grand Valley State University. All rights reserved.

For permissions or questions regarding the use of this material, contact hasanka@gvsu.edu.

Please cite [1] when using, adapting, or reproducing this material. The corresponding BibTeX entry is provided below:

```
@misc{hasan2026stesting,  
author = {Hasan, Kamrul},  
year = {2026},  
title = {Software Testing},  
institution = {Grand Valley State University},  
url = {https://github.com/mdkamrulhasan/software-testing}  
}
```

Contents

1	Learning Objectives	3
2	Recap: From Test Doubles to Mocking	3
3	Why Real Dependencies Get in the Way	3
4	Creating and Configuring a Mock	4
5	Simulating Failures with side_effect	4
6	Verifying Interactions	5
7	Mock vs. MagicMock: Why Two Classes?	5
7.1	Example: A Richer Database Interface	6
7.2	Example: A Context-Manager Cursor	6
7.3	A Decision Rule	7
8	Constraining Mocks with spec and autospec	7
9	Replacing Dependencies with patch()	8
10	When to Mock, and When Not To	9

11 Common Mistakes	9
12 A Practical Mental Model	10
13 Summary	10
14 Practice Exercises	11
15 Discussion Questions	12
16 Key Takeaways for SE 413/513	12
17 References	12

1 Learning Objectives

After completing this reading, you should be able to:

- Explain why real dependencies make unit tests slow, fragile, or hard to control, and how mocking addresses this.
- Create and configure `Mock` and `MagicMock` objects using `return_value` and `side_effect`.
- Verify how a dependency was used with call assertions and call-introspection attributes.
- Explain the difference between `Mock` and `MagicMock`, and recognize when magic-method support actually matters.
- Use `spec` and `autospec` to keep a mock honest about the interface it replaces.
- Use `patch()` to replace a dependency during a test, and explain why the *patch target* matters.
- Recognize common mocking mistakes, including over-mocking and interaction assertions that make tests fragile.

2 Recap: From Test Doubles to Mocking

The Week 4 reading introduced five kinds of **test double** — dummy, stub, fake, spy, and mock — using a running `order_service` example with three collaborators: a **payment service**, a **database**, and an **email service**. It also introduced **state verification** versus **behavior verification**, and the classical (Detroit) versus mockist (London) schools of TDD.

Key Idea

This reading assumes that vocabulary. If dummy, stub, fake, spy, mock, or “state vs. behavior verification” do not sound familiar, revisit the Week 4 reading before continuing — we will not re-derive them here.

This week goes deeper into *mocking* specifically: the mechanics of Python’s built-in `unittest.mock` module[2], the two mock classes it provides (`Mock` and `MagicMock`), and `patch()`, the tool most students will actually reach for when a dependency cannot simply be passed in through a constructor.

3 Why Real Dependencies Get in the Way

A unit test should exercise a small piece of behavior in isolation. Real dependencies routinely make that difficult, because they can be:

- **slow** — a network round-trip or disk access on every test run;
- **expensive** — a real payment charge, a paid API call;
- **nondeterministic** — the current time, random values, race conditions;
- **external** — outside your control, and possibly unavailable in CI; or
- **side-effecting** — sending a real email, writing a real file.

Typical examples: REST APIs, payment providers, databases, message queues, file systems, and third-party SDKs — the same categories of hard-coded dependency the Week 4 reading identified as candidates for dependency injection and test doubles[7]. Mocking is how you replace these dependencies with an object you fully control, while still exercising the real code that talks to them.

4 Creating and Configuring a Mock

Recall `place_order()` from the Week 4 reading:

```
def place_order(payment_service):
    if payment_service.process_payment(100):
        return "Order confirmed"
    return "Payment failed"
```

Week 4 used a bare `Mock()` only to verify that `process_payment(100)` was called. A mock can do more: it can also be *configured* to return whatever value the test needs.

```
from unittest.mock import Mock

payment_service = Mock()
payment_service.process_payment.return_value = True

result = place_order(payment_service)

assert result == "Order confirmed"
payment_service.process_payment.assert_called_once_with(100)
```

The test has three recognizable steps: **(1)** create the mock, **(2)** configure its behavior, **(3)** pass it to the code under test. This pattern recurs throughout mocking, regardless of which mock class or configuration option is used.

Tip

`return_value` does not depend on the arguments a mock was called with — `payment_service.process_payment(100)` and `payment_service.process_payment(99999)` return exactly the same configured value. If a test needs different results for different inputs, use `side_effect` (Section 5) instead.

5 Simulating Failures with `side_effect`

Real payment services fail sometimes, and error-handling code deserves tests just as much as the happy path does. `side_effect` lets a mock raise an exception instead of returning a value:

```
payment_service = Mock()
payment_service.process_payment.side_effect = ConnectionError(
    "payment gateway unreachable"
)

def test_place_order_handles_gateway_failure():
    try:
        place_order(payment_service)
        assert False, "expected a ConnectionError"
    except ConnectionError:
        pass
```

`side_effect` can also be a function, which the mock calls instead of returning a canned value — useful when the result should depend on the input:

```
def fee_for(amount):
```

```

    return amount * 0.03

payment_service.calculate_fee.side_effect = fee_for
assert payment_service.calculate_fee(100) == 3.0

```

Or a list, consumed one value per call — handy for simulating a flaky dependency that fails once and then succeeds:

```

payment_service.process_payment.side_effect = [
    ConnectionError("timeout"),
    True,
]

```

6 Verifying Interactions

A mock also *remembers* how it was used, which is what makes behavior verification possible. Beyond `assert_called_once_with(...)` from Week 4, a few attributes are useful for inspecting calls directly:

Expression	Purpose
<code>.assert_called()</code>	Verify the mock was called at least once
<code>.assert_called_once()</code>	Verify it was called exactly once (any arguments)
<code>.assert_called_once_with(...)</code>	Verify exactly one call, with these arguments
<code>.call_count</code>	How many times it was called
<code>.call_args</code>	The arguments of the <i>most recent</i> call
<code>.mock_calls</code>	The full sequence of calls made on the mock

```

payment_service = Mock()
payment_service.process_payment.return_value = True

place_order(payment_service)

assert payment_service.process_payment.call_count == 1
print(payment_service.process_payment.call_args)    # call(100)

```

Common Pitfall

Do not verify every interaction simply because a mock makes it possible. A test that asserts on every call a mock received couples itself to *how* the code is implemented, not *what* it does — this is the same fragility Week 4 discussed under the mockist school's trade-offs[4]. Assert the interactions that matter to the behavior you are testing, and let the rest go unchecked.

7 Mock vs. MagicMock: Why Two Classes?

Everything so far has used `Mock`, and it has been enough — because `process_payment(100)` is an **ordinary method call**. But Python has an entire category of behavior that is *not* an ordinary method call: operations like `len(obj)`, `obj[0]`, `item in obj`, iteration, and the `with` statement are implemented through **magic methods** (also called *dunder methods*, for their double-underscore names)[3].

Operation	Underlying magic method
<code>len(obj)</code>	<code>obj.__len__()</code>
<code>obj[key]</code>	<code>obj.__getitem__(key)</code>
<code>item in obj</code>	<code>obj.__contains__(item)</code>
<code>for x in obj</code>	<code>obj.__iter__()</code>
<code>with obj:</code>	<code>obj.__enter__()</code> and <code>obj.__exit__()</code>

A plain `Mock` does not implement these magic methods — calling `len()` on one raises a `TypeError`. `MagicMock` is a subclass of `Mock` that pre-configures all of them, so they work out of the box and can be configured just like any other mocked method.

Key Idea

Mock handles ordinary method calls. **MagicMock** handles ordinary method calls *and* Python’s magic-method protocols. Neither is “better” — the right choice depends entirely on the interface the code under test actually uses.

7.1 Example: A Richer Database Interface

Week 4’s `FakeDatabase` was a simple wrapper around a dictionary, with two ordinary methods, `save_user()` and `get_user()` — a plain `Mock` (or a `fake`) is a perfectly natural fit for that interface. But real database interfaces are often richer. Suppose `order_service`’s database object also supports being treated like a collection of orders:

```
def summarize_orders(database):
    if len(database) == 0:
        return "No orders yet"
    return f"{len(database)} orders, most recent: {database[0]}"
```

`summarize_orders()` uses `len(database)` and `database[0]` — both magic-method operations. A `MagicMock` models this directly, without writing a fake collection class:

```
from unittest.mock import MagicMock

database = MagicMock()
database.__len__.return_value = 3
database.__getitem__.return_value = "Order #1042"

assert summarize_orders(database) == "3 orders, most recent: Order #1042"
```

Membership and iteration follow the same pattern:

```
database.__contains__.return_value = True
assert "customer_42" in database

database.__iter__.return_value = iter(["Order #1040", "Order #1041"])
assert list(database) == ["Order #1040", "Order #1041"]
```

7.2 Example: A Context-Manager Cursor

Many real database libraries expose a cursor as a context manager:

```
def get_customer_name(database, order_id):
    with database.connection() as cursor:
```

```

        cursor.execute("SELECT customer FROM orders WHERE id = ?", (order_id,))
        row = cursor.fetchone()
    return row[0]

```

The `with` statement relies on `__enter__()` and `__exit__()`. A `MagicMock` makes this straightforward to set up:

```

database = MagicMock()
cursor = database.connection.return_value.__enter__.return_value
cursor.fetchone.return_value = ("Alice",)

assert get_customer_name(database, 1042) == "Alice"
cursor.execute.assert_called_once_with(
    "SELECT customer FROM orders WHERE id = ?", (1042,)
)

```

Tip

Read `database.connection.return_value.__enter__.return_value` from left to right: “calling `database.connection()` returns *this*; entering *that* as a context manager returns *this*.” Nested mock configuration always mirrors the chain of calls the production code actually makes.

7.3 A Decision Rule

Question

Does the code only call ordinary methods on the dependency?

→ Use `Mock`.

Does the code use `len()`, indexing, iteration, membership testing (`in`), or a `with` block on the dependency?

→ Use `MagicMock`.

Not sure yet, or the interface is mixed?

→ `MagicMock` is a safe default — it supports ordinary methods too — but prefer `Mock` when it is sufficient, since it says more clearly what the test actually needs (Section 11).

8 Constraining Mocks with `spec` and `autospec`

An unconstrained mock accepts *any* attribute access, which hides a real class of bugs:

```

payment_service = Mock()
payment_service.process_payment(100)    # typo -- but Mock allows it silently

```

If the real `PaymentService` has no `process_payment` method, this test would pass while the equivalent production call would raise an `AttributeError`. Passing `spec` constrains the mock to the real class’s interface:

```

from unittest.mock import Mock

class PaymentService:
    def process_payment(self, amount):
        ...

payment_service = Mock(spec=PaymentService)
payment_service.process_payment(100)    # OK

```

```
payment_service.process_payment(100)    # raises AttributeError
```

`autospec` (most often seen as `autospec=True` with `patch()`, Section 9) goes further: it also checks *call signatures*, so calling a one-argument method with two arguments fails the same way it would against the real class[2].

Tip

`spec/autospec` do not make a mock “more correct” about behavior — they only catch interface mismatches (wrong method name, wrong number of arguments). They are cheap insurance and a good default habit, especially on a mock that stands in for a class with a nontrivial interface.

9 Replacing Dependencies with `patch()`

Constructor injection (Week 4) works when *you* control how a collaborator is supplied. It does not work for a module-level function called directly, such as `requests.get`. Suppose `order_service` checks shipping status through an external API:

```
# shipping.py
import requests

def get_shipping_status(order_id):
    response = requests.get(f"https://ship.example.com/track/{order_id}")
    return response.json()["status"]
```

A unit test should not make a real HTTP request. `unittest.mock.patch()` temporarily replaces an object for the duration of a test:

```
from unittest.mock import patch

@patch("shipping.requests.get")
def test_get_shipping_status(mock_get):
    mock_get.return_value.json.return_value = {"status": "In Transit"}

    status = get_shipping_status(1042)

    assert status == "In Transit"
    mock_get.assert_called_once_with(
        "https://ship.example.com/track/1042"
    )
```

Key Idea

Patch the name where it is *looked up*, not where it was originally defined. `shipping.py` does `import requests` and then calls `requests.get(...)`, so `shipping` is where the name `requests.get` is looked up at call time — hence `@patch("shipping.requests.get")`, not `@patch("requests.get")`. This is one of the most common sources of confusion when first learning `patch()`.

By default, `patch()` replaces the target with a `MagicMock` — which is why chaining `mock_get.return_value.json.return_value` works without any extra setup: `mock_get`, its `return_value`, and every attribute accessed on it are all `MagicMock` objects unless configured otherwise. Passing `autospec=True` to `patch()`

additionally constrains the replacement to the real function’s signature, combining patching with the interface safety of Section 8.

10 When to Mock, and When Not To

Mocking is not automatically the right choice. Consider a small, deterministic, in-process helper:

```
def apply_discount(amount, rate):  
    return amount * (1 - rate)
```

There is little to gain from mocking `apply_discount()` in a test of code that calls it — it is fast, has no external effects, and a real call is simpler than any mock configuration.

The general principle: mock dependencies that are slow, external, nondeterministic, or expensive; use the real object for everything else[4, 5]. This is the same guidance the Week 4 reading gave for choosing between the classical and mockist styles of testing — mocking mechanics do not change that judgment, they just give you the tools to act on it.

Common Pitfall

Recall the **state vs. behavior verification** trade-off from Week 4: a test that asserts on the *result* of `place_order()` survives a refactor that preserves behavior; a test that over-asserts on *how* `payment_service` was called can break on a harmless internal change. The tools in this reading (`return_value`, `side_effect`, call assertions) can be used either way — the mocking library does not make this decision for you.

11 Common Mistakes

Common Pitfall

Patching the wrong location. Patch the name as it is looked up by the module under test (Section 9), not where the dependency was originally defined.

Common Pitfall

Over-specifying interactions. Asserting on every call a mock received makes a test brittle to harmless refactors (Section 6). Assert what matters to the behavior; let the rest go unchecked.

Common Pitfall

Skipping spec/autospec. An unconstrained mock hides typos and interface drift that would fail loudly against the real class.

Common Pitfall

Using MagicMock everywhere out of habit. If the code under test only makes ordinary method calls, a plain `Mock` says so more clearly — reach for `MagicMock` because the interface needs magic methods, not by default.

Common Pitfall

Testing the mock instead of the code. A test consisting only of `mock.some_method.assert_called_once()` says little about whether the application actually behaves correctly. Pair interaction assertions with an assertion on an observable result whenever one exists.

12 A Practical Mental Model

When you see `Mock()` or `MagicMock()`, think:

“A programmable stand-in whose behavior I control and whose interactions I can inspect afterward.”

When you see `mock.method.return_value = ...`, think:

“When the code calls this, hand it this result.”

When you see `mock.method.side_effect = ...`, think:

“Make this interaction fail, or compute its result, in a way I control.”

When you see `mock.__len__`, `mock.__getitem__`, or similar, think:

“The code is using a Python protocol here, not an ordinary method — this needs `MagicMock`.”

When you see `@patch("module.name")`, think:

“Replace this name, where *this module* looks it up, for the duration of the test.”

13 Summary

Expression	Purpose
<code>Mock()</code>	A configurable test double for ordinary method calls
<code>MagicMock()</code>	A <code>Mock</code> that also supports Python magic methods
<code>return_value</code>	Configure what a call returns
<code>side_effect</code>	Raise an exception, call a function, or return values from a sequence
<code>assert_called_once_with(...)</code>	Verify one call, with these arguments
<code>call_count</code> / <code>call_args</code> / <code>mock_calls</code>	Inspect how a mock was used
<code>spec</code>	Constrain a mock to a real class’s attributes
<code>autospec</code>	<code>spec</code> , plus call-signature checking
<code>patch()</code>	Temporarily replace a dependency for a test

	Mock	MagicMock
Ordinary methods, <code>return_value</code> , <code>side_effect</code> , assertions	Yes	Yes
<code>len(obj)</code> , <code>obj[key]</code> , <code>in</code> , iteration, <code>with</code>	No (raises <code>TypeError</code>)	Yes
Best when	Interface is plain method calls	Interface uses a Python protocol

Key Idea

Mock the dependency; test the behavior. Choose the simplest tool — Mock over MagicMock, a targeted `side_effect` over a pile of interaction assertions — that accurately represents the interface the code under test actually needs.

14 Practice Exercises

Exercise 1 — Basic Configuration

Given `place_order()` from this reading, write a test in which `process_payment` raises `ConnectionError`, and verify that `place_order()` propagates it (or, if you first add error handling to `place_order()`, that it returns an appropriate failure message instead).

Exercise 2 — Choosing Mock or MagicMock

For each situation, decide whether Mock or MagicMock is the more natural choice and justify your answer:

1. An `EmailService` exposes `send(to, message)`.
2. A `ProductCatalog` is accessed using `catalog[sku]`.
3. A collection of shipping events is processed with `for event in events`.
4. A `Logger` exposes `logger.info(message)`.
5. A database connection is used with `with connection:`.

Exercise 3 — Interface Safety

Take the `PaymentService` class from Section 8. Create a `Mock(spec=PaymentService)` and demonstrate: (a) that `process_payment(100)` succeeds, and (b) that calling a misspelled method raises `AttributeError`.

Exercise 4 — Patching an External Call

Using `get_shipping_status()` from Section 9, write a test with `@patch()` that simulates the shipping API returning `{"status": "Delivered"}`, and verify both the returned status and the URL that was requested.

Exercise 5 — Design Question

A teammate says: “Since `MagicMock` can do everything `Mock` can do, we should just always use `MagicMock`.” Using at least one concept from this reading, argue for or against this position.

15 Discussion Questions

1. Why is `return_value` not sensitive to the arguments a mock was called with, and when does that matter?
2. When would `side_effect` be preferable to `return_value`?
3. Why can excessive call-assertion checking make a test fragile, even when the production code is correct?
4. Why does a plain `Mock` raise a `TypeError` on `len()`, while a `MagicMock` does not?
5. What specific problem does `spec` solve that an unconstrained mock cannot?
6. Why is the `location` passed to `patch()` more important than where a dependency was originally defined?
7. How does the `Mock`/`MagicMock` distinction relate to the state-vs-behavior-verification distinction from Week 4? Are they the same choice, or independent ones?
8. In what situation would you prefer a real object over any kind of mock, even though mocking is possible?

16 Key Takeaways for SE 413/513

1. **Mocking replaces slow, external, or nondeterministic dependencies** so a unit test can run fast and deterministically.
2. **`return_value` and `side_effect` configure behavior**; call assertions and `call_args/mock_calls` verify interactions.
3. **`Mock` handles ordinary methods; `MagicMock` adds Python's magic methods** — choose based on the interface the code under test actually uses, not by default.
4. **`spec` and `autospec` catch interface mismatches** that an unconstrained mock would silently hide.
5. **`patch()` replaces a name where it is looked up**, not where it was defined — the single most common source of confusion for new users.
6. **Mocking mechanics do not replace judgment**. Whether to mock at all, and whether to verify state or behavior, are the same design questions raised in Week 4.

17 References

- Python's `unittest.mock` library, and the mechanics of `Mock`, `MagicMock`, `spec/autospec`, and `patch()`.^[2]
- Python's data model and the special (magic/dunder) method names that `MagicMock` implements.^[3]
- The distinction between mocks and other test doubles, and state vs. behavior verification.^[4, 5]
- Interaction-based (mockist) testing in practice, introduced in the Week 4 reading.^[6]
- Using `unittest.mock` together with `pytest`, this course's default testing framework.^[8]
- Course material on software testing principles and related testing practices.^[1]

References

- [1] Hasan, K. (2026). *Software Testing*. Grand Valley State University. <https://github.com/mdkamrulhasan/software-testing>
- [2] Python Software Foundation. (2026). *unittest.mock* — mock object library. <https://docs.python.org/3/library/unittest.mock.html>
- [3] Python Software Foundation. (2026). *3. Data Model*. The Python Language Reference. <https://docs.python.org/3/reference/datamodel.html>

- [4] Fowler, M. (2007). *Mocks Aren't Stubs*. martinowler.com. <https://martinfowler.com/articles/mocksArentStubs.html>
- [5] Meszaros, G. (2007). *xUnit Test Patterns: Refactoring Test Code*. Addison-Wesley.
- [6] Freeman, S., & Pryce, N. (2009). *Growing Object-Oriented Software, Guided by Tests*. Addison-Wesley Professional.
- [7] Feathers, M. C. (2004). *Working Effectively with Legacy Code*. Prentice Hall.
- [8] Okken, B. (2022). *Python Testing with pytest* (2nd ed.). Pragmatic Bookshelf.